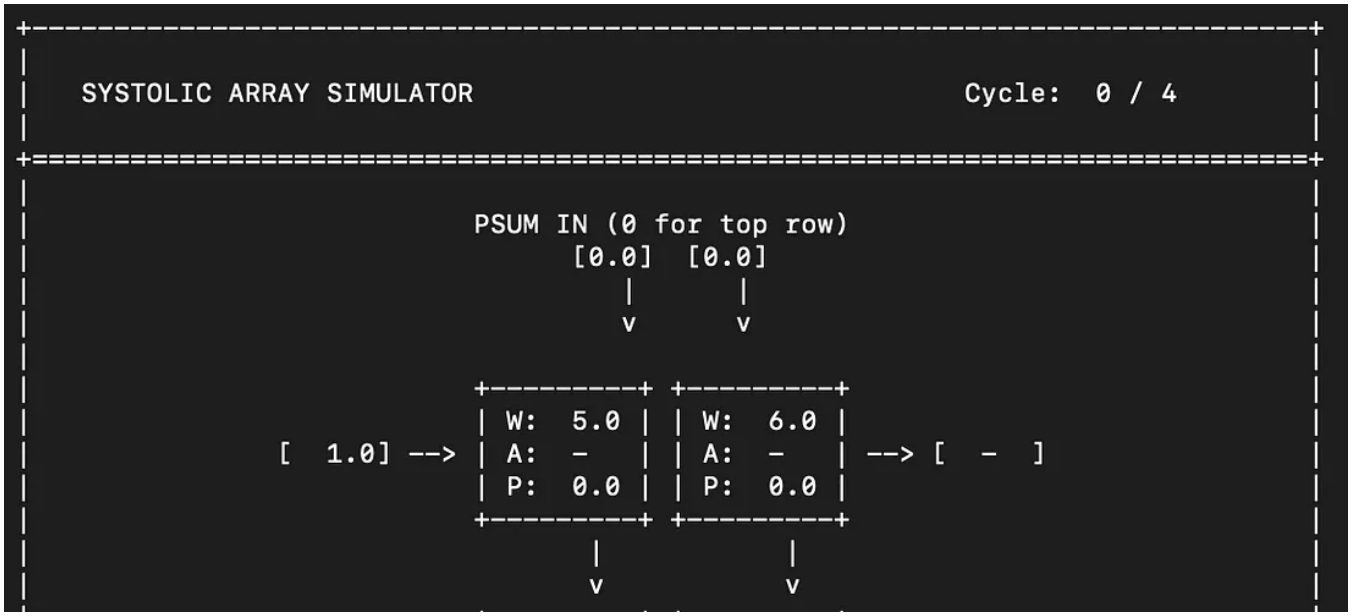
Open in app ↗

Under the Hood of MatMuls: A Rust Simulation of the TPU's Systolic Array

18 min read · Dec 11, 2025


Owumi Festus Follow

Listen Share More



*"**Note:** This simulator models only the core data-flow of a weight-stationary systolic array, similar to the matrix-multiply unit (MXU) inside a real TPU.* Actual TPU *hardware embeds that array inside a much larger architecture which includes memory-hierarchy logic, tiling or buffer management, instruction scheduling and control logic, none of which are simulated here. Also, while this simulator uses 32-bit floats for clarity and simplicity, many modern TPUs perform matrix multiplications in lower-precision formats for inputs and then accumulate results in 32-bit float to improve performance and reduce memory/ bandwidth demands. (see Google's TPU architecture overview) Finally, in real hardware,*

all PEs compute simultaneously each cycle. In this Rust simulator, we emulate this behavior sequentially for clarity."

Hello World!

It's nice to be able to write again and share small pieces of information that could be helpful to anyone looking to implement a new solution, learn a new skill, or just demystify the magic behind the tools we use every day.

Today we're inspired by Alan Ma and Abiral Shakya. They kicked off a project called "Unwrapping TPUs" on their Substack, *Chewing on Chips:* 31 days of figuring out how TPUs actually work, from the ground up. Systolic arrays. MatMuls. Pipelining. ISA breakdowns. The whole stack. I was following along, and somewhere around Day 2 or 3, I realized I wanted to build something to understand the content better.

Every framework hides the details of MatMuls. NumPy hides it. PyTorch hides it. Even CUDA hides it behind kernels. But when you look closely at accelerators like Google's TPU, they don't treat matmul as "a loop." They treat it as a dataflow problem. The computation moves, not the instructions. I wanted to understand that properly, by building something I could step through cycle by cycle. So I wrote a software simulator in Rust that behaves like a systolic array: the piece of hardware inside the TPU responsible for almost all of its math.

If you want to follow Alan and Abiral's work, check out Chewing on Chips.

Let's take a look under the hood.

Why Google Built the TPU v1 (and Why MatMul Needed a New Kind of Hardware)

Before talking about systolic arrays, it's worth understanding why Google built the TPU v1 in the first place. The story wasn't "we needed faster matrix multiplies." It's bigger than that.

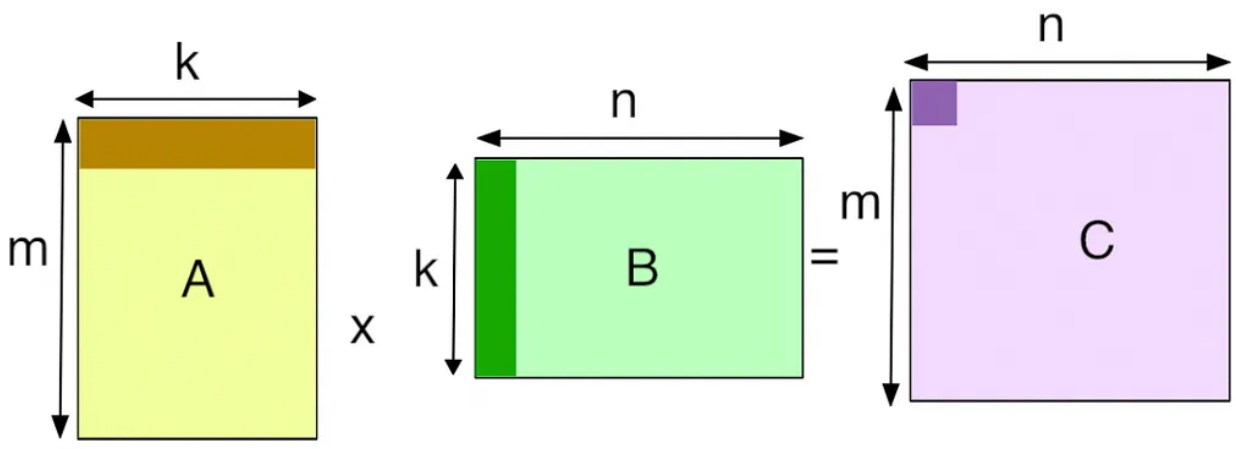
In 2013, Google realized something alarming: if neural networks kept scaling the way they were, the company would eventually need to double its datacenters just to keep up with inference loads across Search, Street View, Translate, Photos, and

everything else that relied on neural nets. And the problem wasn't floating-point compute because the industry already had powerful CPUs and GPUs. NVIDIA's accelerators were everywhere in deep learning research. The real bottleneck was: Running neural networks at scale was becoming unsustainably expensive, memory-bound, and too power-hungry.

Neural network inference is dominated by one operation:

$$C=A\times B$$

```
for i in 0..m {
    for j in 0..n {
        for k in 0..K {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```



https://petewarden.com/2015/10/25/an-engineers-guide-to-gemm/

a giant matrix multiply between activations and weights.

But on CPUs and GPUs, even with highly optimized libraries, the bottleneck is still the data movement pattern:
- the same weights get fetched again and again (especially on CPUs; GPUs mitigate this with tiling and on-chip memory, but they're still often bandwidth-bound),
- the same activations are repeatedly pulled from memory,
- partial sums bounce in and out of registers and caches, and the chip burns more energy moving data than computing on it.

So they asked a different question: What if we could keep weights on-chip, stream activations through once, and reuse everything?

This is the moment where the TPU v1's design becomes obvious in hindsight:

- quantized 8-bit multipliers instead of expensive FP32 units
- 65,536 Multiply–Accumulate (MAC) units arranged in a 256×256 grid
- systolic dataflow instead of general-purpose control logic
- 24MB of on-chip SRAM to hold activations and partial sums

The MXU (Matrix Multiply Unit) behaves like a matrix processor: instead of performing operations one scalar at a time (CPU), or thousands at a time (GPU), the TPU v1 performs hundreds of thousands of operations every single cycle through a systolic array.

And because the architecture reuses data in place, the TPU doesn't need:

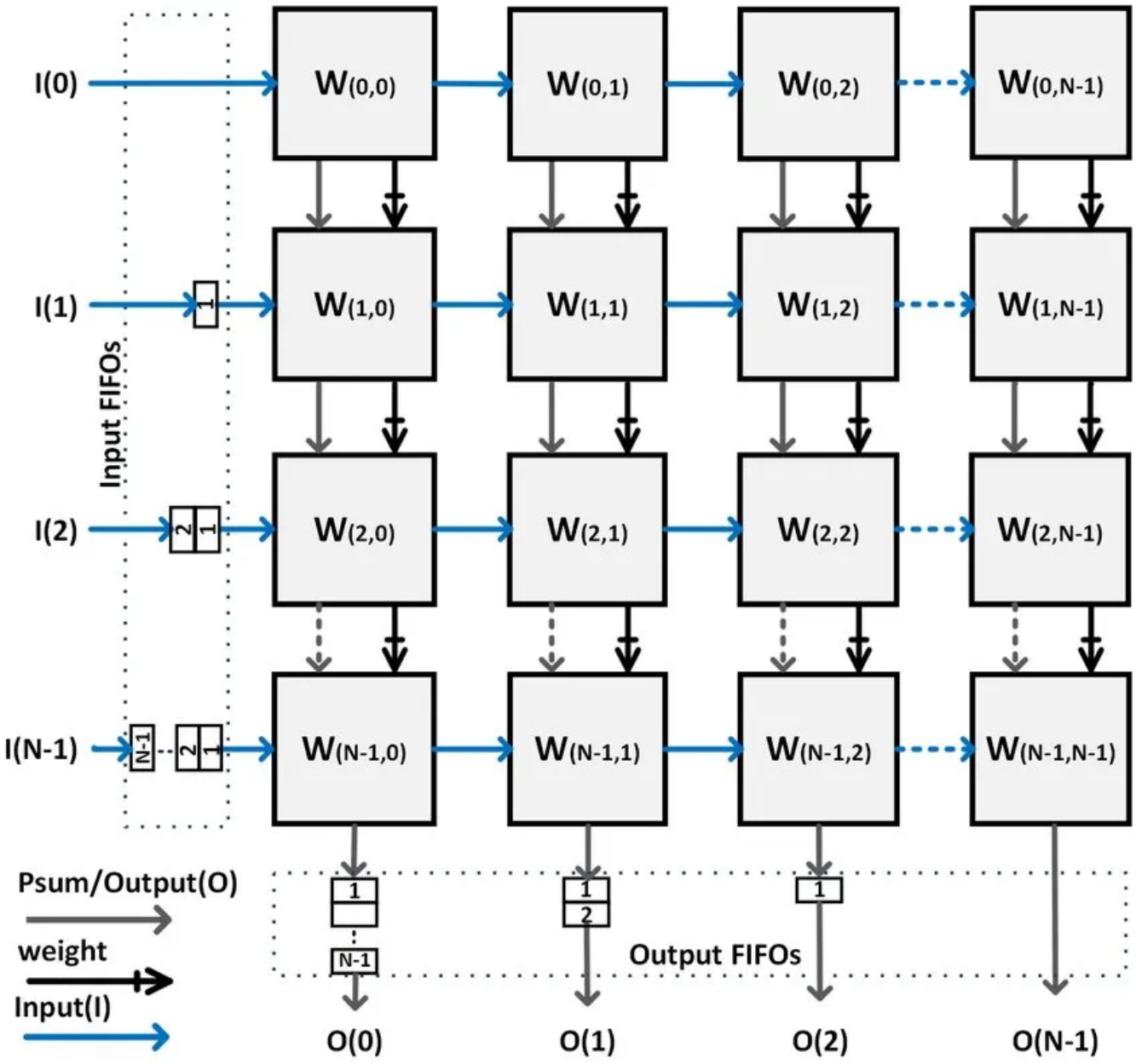
- caches
- branch predictors
- out-of-order execution
- warp schedulers
- speculative prefetching

and all the heavyweight features that make CPUs and GPUs unpredictable and power-hungry.

This brings us naturally to the heart of the TPU v1: a 256×256 systolic array that performs 65,536 multiply-accumulates every cycle at 700 MHz.

Now let's look at how that works under the hood.

What Is a Systolic Array?



Top-level schematic for NxN weight stationary systolic array (https://www.researchgate.net/

publication/387078780_DiP_A_Scalable_Energy-Efficient_Systolic_Array_for_Matrix_Multiplication_Acceleration)

A systolic array is a 2D grid of tiny compute cells called Processing Elements (PEs). Each one holds a weight. Activations enter from the left. Partial sums enter from the top. Every cycle, each cell does one thing:

```
psum_out = psum_in + (act_in * weight)
```

Then:

- The activation moves to the right.
- The partial sum moves down.
- The weight stays where it is.

This is called weight-stationary dataflow. The weights don't move, they sit in place while data flows around them.

If you arrange these cells in a grid, something interesting happens: a diagonal wavefront of computation forms. Values ripple through the array like a wave, and the final results appear at the bottom row after a fixed number of cycles.

Here's what a 2×2 array looks like:

```
          psum_in=0      psum_in=0
              ↓              ↓
act_in → [ PE(0,0) ] → [ PE(0,1) ] → (act discarded)
              ↓              ↓
act_in → [ PE(1,0) ] → [ PE(1,1) ] → (act discarded)
              ↓              ↓
          psum_out       psum_out
          (result)       (result)
```

Each arrow is one clock cycle of delay. Activations march rightward. Partial sums accumulate downward. By the time data reaches the bottom, the dot products are complete.

Data flowing through a grid of multiply-accumulate units. This is why the TPUs scale so well. The same pattern works whether you have a 2×2 array or a 256×256 array. More cells, same structure.

The Processing Element in Rust

The PE is the smallest piece of the systolic array. Get this right, and the rest is just wiring.

A single PE does three things every clock cycle:

1. Multiply-accumulate: Take the incoming activation, multiply it by the stored weight, add it to the incoming partial sum.
2. Pass the activation to the right: The same value that came in goes out to the next PE in the row.
3. Pass the partial sum downward: The result of the MAC goes to the next PE in the column.

The weight never moves. It gets loaded once and stays put. That's the "weight-stationary" part.

One detail worth noting: not every cycle has valid data. Sometimes a PE is waiting for the wavefront to reach it. I use Option<f32> for the activation to make that explicit: Some(value) when there's real data, None when the PE should just pass things through.

Here's the full implementation:

```
/// https://github.com/Ayobami-00/tpu-matmul-simulator-rs/blob/main/src/pe.rs
/// A single Processing Element in a weighted-stationary systolic array.
```

```rust
#[derive(Debug, Clone)]
pub struct PE {
    weight: f32,
}

impl PE {
    /// Create a new PE with the given weight.
    pub fn new(weight: f32) -> Self {
        Self { weight }
    }

    /// Execute one clock cycle.
    ///
    /// - `act_in`: activation from the left (or None if no data this cycle)
    /// - `psum_in`: partial sum from above
    ///
    /// Returns `(act_out, psum_out)`:
    /// - `act_out`: activation to pass rightward
    /// - `psum_out`: partial sum to pass downward
    pub fn step(&self, act_in: Option<f32>, psum_in: f32) -> (Option<f32>, f32)
        match act_in {
            Some(a) => {
                let psum_out = psum_in + a * self.weight;
                (Some(a), psum_out)
            }
            None => {
                // No activation this cycle; just pass the partial sum through
                (None, psum_in)
            }
        }
    }
}
```

Building the Systolic Array

Now we connect multiple PEs into a grid. The systolic array is just a 2D collection of PEs with a specific wiring pattern:

- Activations flow left → right (horizontally)
- Partial sums flow top → bottom (vertically)
- Each PE holds one weight from the weight matrix

The tricky part isn't the PEs themselves, it's managing the state between cycles. At

any given moment, we need to track:

1. Where each activation currently sits in the grid
2. What partial sum is entering each PE from above

I represent this with two state matrices: act_state for activations and psum_state for partial sums. Each cycle, we compute the next state based on the current one.

The tick() function runs one clock cycle across the entire grid. It takes:

- The current activation state
- The current partial sum state
- New activations entering from the left edge

And it returns:

- The next activation state
- The next partial sum state
- The outputs emerging from the bottom row

Here's the implementation:

```
/// https://github.com/Ayobami-00/tpu-matmul-simulator-rs/blob/main/src/array.r
use crate::pe::PE;

/// A 2D systolic array of Processing Elements.
#[derive(Debug)]
pub struct SystolicArray {
    pub rows: usize,
    pub cols: usize,
    pes: Vec<Vec<PE>>,
}

impl SystolicArray {
    /// Create a systolic array from a weight matrix.
    ///
```

```rust
    /// `weights[r][c]` becomes the weight for PE at position (r, c).

    pub fn new(weights: Vec<Vec<f32>>) -> Self {
        let rows = weights.len();
        let cols = if rows > 0 { weights[0].len() } else { 0 };

        let pes = weights
            .into_iter()
            .map(|row| row.into_iter().map(PE::new).collect())
            .collect();

        Self { rows, cols, pes }
    }

    /// Execute one clock cycle across the entire array.
    ///
    /// # Arguments
    /// - `act_state`: Current activation at each PE position (from previous cy
    /// - `psum_state`: Current partial sum at each PE position (from previous
    /// - `left_inputs`: New activations entering each row from the left
    ///
    /// # Returns
    /// - `next_act_state`: Activation state for the next cycle
    /// - `next_psum_state`: Partial sum state for the next cycle
    /// - `bottom_outputs`: Partial sums exiting the bottom row
    pub fn tick(
        &self,
        act_state: &[Vec<Option<f32>>], // where activations currently sit
        psum_state: &[Vec<f32>],  // partial sums from last cycle
        left_inputs: &[Option<f32>], // new activations entering from left
    ) -> (Vec<Vec<Option<f32>>>, Vec<Vec<f32>>, Vec<f32>) {
        let mut next_act = vec![vec![None; self.cols]; self.rows];
        let mut next_psum = vec![vec![0.0; self.cols]; self.rows];
        let mut bottom_outputs = vec![0.0; self.cols];

        for r in 0..self.rows {
            for c in 0..self.cols {
                // Activation input: from left edge or from the PE to our left
                let act_in = if c == 0 {
                    left_inputs[r] // leftmost column gets fresh input
                } else {
                    act_state[r][c - 1] // others get from the PE to their left
                };

                // Partial sum input: zero at top row, otherwise from PE above
                let psum_in = if r == 0 { 0.0 } else { psum_state[r - 1][c] };

                // Run this PE for one cycle
```

```
                let (act_out, psum_out) = self.pes[r][c].step(act_in, psum_in);

                // Store activation for next cycle (will be read by PE to our r
                next_act[r][c] = act_out;

                // Store or output partial sum
                if r == self.rows - 1 {
                    // Bottom row: output exits the array
                    bottom_outputs[c] = psum_out;
                }
                // Always store psum for next cycle (used by PE below us)
                next_psum[r][c] = psum_out;
            }
        }

        (next_act, next_psum, bottom_outputs)
    }

    /// Get array dimensions.
    pub fn dimensions(&self) -> (usize, usize) {
        (self.rows, self.cols)
    }
}
```

Visualizing the Wavefront

At this point we have a working systolic array, but it's hard to see what's actually happening inside. The whole point of this project was to make matmul visible, so let's build a real-time visualization.

The goal: watch data flow through the array, cycle by cycle, like a slow-motion replay of what happens inside a TPU.

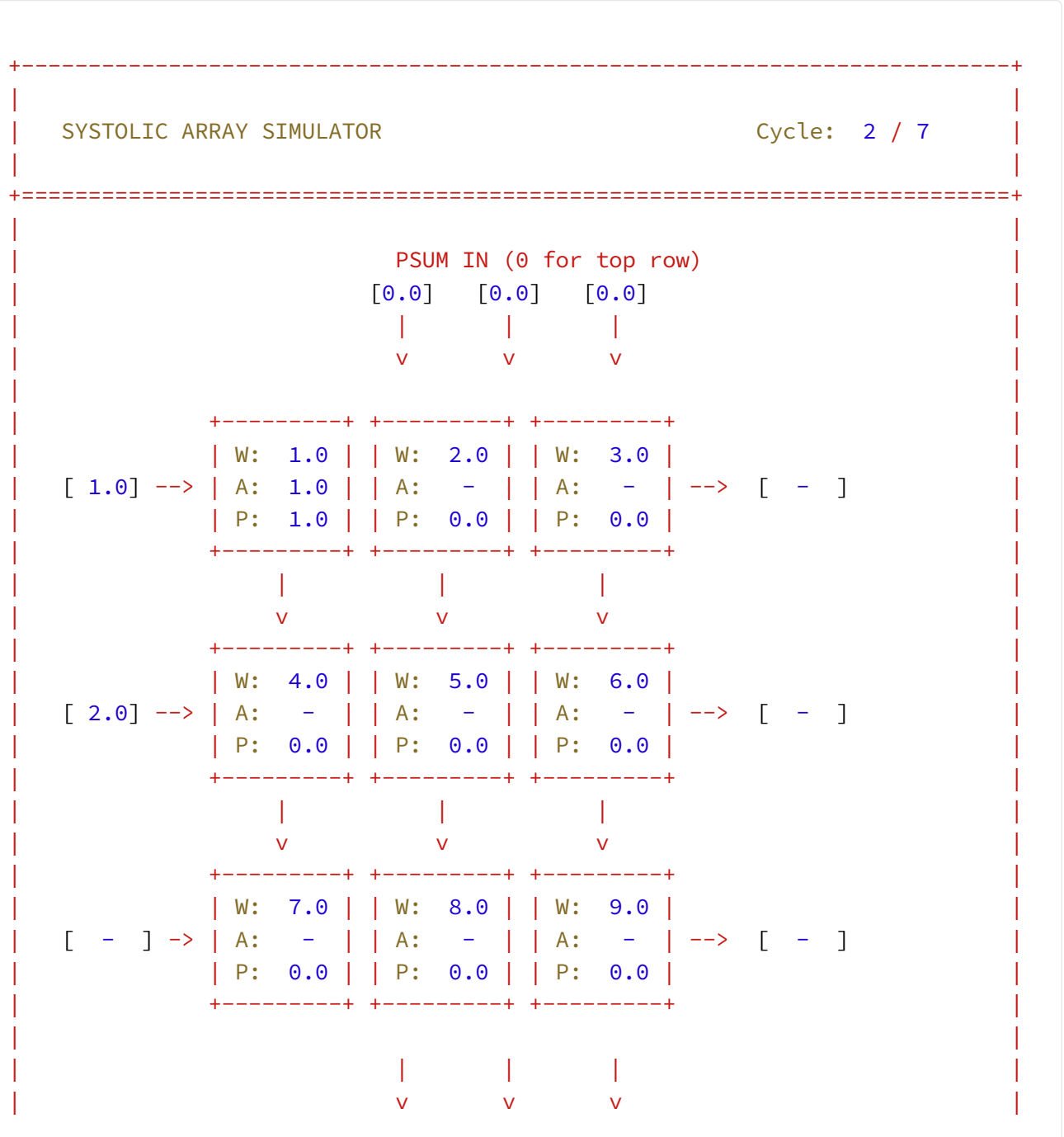
Here's what we want to show:

- Activations entering from the left: the input values waiting to be fed into each row
- The grid itself: each PE displaying its weight, current activation, and partial sum
- Activations exiting on the right: values that have passed through the row
- Partial sums exiting at the bottom: the actual outputs of the computation

Every cycle, we clear the terminal and redraw the entire state. With a short delay between frames, you get an animation. The diagonal wavefront becomes obvious and you can literally see activations march rightward while partial sums accumulate downward.

Here's what a single frame looks like for a 3×3 array mid-computation:

You can see the visualizer code here.



```
|                          [0.0]   [0.0]   [0.0]                          |
|                            PSUM OUT (result)                            |
|                                                                         |
+=========================================================================+
|   W = weight     A = activation     P = partial sum                     |
|   Delay: 300ms/cycle                                                    |
+-------------------------------------------------------------------------+
```

What Each Part Shows

- PSUM IN — Partial sums entering from the top (always 0 in weight-stationary)
- [1.0] → — Activation entering this row from the left
- W: 1.0 — Weight stored in this PE
- A: 1.0 — Activation currently at this PE
- P: 1.0 — Partial sum computed so far at this PE
- → [—] — Activation exiting the right edge (discarded)
- v — Data flow direction (downward)
- PSUM OUT — Final results exiting the bottom row

Note: PSUM IN is zero for the top row only. Lower rows receive partial sums from the PEs above them.

Driving MatMul Through the Array

Now we have a systolic array and a way to visualize it. But we haven't actually done a matrix multiplication yet. The array can tick forward cycle by cycle, but something needs to feed it the right data at the right time.

That's what the scheduler does.

A standard matrix multiply looks like this:

```
C = A × B
```

Where:

- `A` is an M×K matrix (the activations)
- `B` is a K×N matrix (the weights)
- `C` is an M×N matrix (the result)

In a systolic array, we don't feed everything in at once. Instead:

1. Weights get loaded into the PEs ahead of time. Each PE holds one element of B. They stay fixed for the entire computation.
2. Activations stream in from the left, one column at a time. But here's the trick: each row is staggered by one cycle. Row 0 starts at cycle 0, row 1 starts at cycle 1, and so on.
3. Results emerge from the bottom after the wavefront passes through. Column 0 finishes first, then column 1, and so on.

The staggering is what creates the diagonal wavefront. Without it, all the data would collide and the math wouldn't work out.

Here's the schedule for a 2×2 matmul with K=2 (two columns in A):

```
Cycle 0:  Row 0 gets A[0][0], Row 1 gets nothing
Cycle 1:  Row 0 gets A[0][1], Row 1 gets A[1][0]
Cycle 2:  Row 0 gets nothing, Row 1 gets A[1][1]
Cycle 3:  Results complete
```

The formula for total cycles is:

```
total_cycles = K + rows + cols - 2
```

For a 2×2 array with K=2: `2 + 2 + 2 - 2 = 4` cycles.

The scheduler handles all of this. It builds the `left_inputs` vector for each cycle, ticks the array, and collects the results from the bottom.

A Note on the Variant Used in This Simulator

The schedule above describes the canonical diagonal systolic matmul, where activations for different rows enter at different cycles to form a full diagonal wavefront.

To keep the visualization simple and intuitive, this simulator uses a closely related variant:

- We compute one output row of C at a time.
- All rows of B are preloaded into the array as stationary weights.
- For a given row C[i,:]C[i, :]C[i,:], we feed the values of A[i,:]A[i, :]A[i,:] into the array one activation per PE row, staggered by one cycle:
- cycle 0 → row 0 receives A[i][0]A[i][0]A[i][0]
- cycle 1 → row 1 receives A[i][1]A[i][1]A[i][1]
- The internal dataflow still forms a diagonal wavefront, but the scheduling is simpler and maps cleanly onto the visualization.

Functionally, this produces the same multiply-accumulate behavior as the textbook systolic design just in a more pedagogically friendly form.

A 2×2 Example

Time to see it all work together.

We'll create a simple example that multiplies two 2×2 matrices and shows the visualization in real time. This is the "hello world" of systolic array matmul which is small enough to follow by hand, but complete enough to prove the concept.

Here's what we're computing:

```
A = | 1  2 |      B = | 5  6 |
    | 3  4 |          | 7  8 |

C = A × B = | 1*5 + 2*7   1*6 + 2*8 |   =   | 19  22 |
            | 3*5 + 4*7   3*6 + 4*8 |       | 43  50 |
```

The systolic array will:

1. Load B as weights into the PEs
2. Stream A's values from the left with staggered timing
3. Accumulate partial sums as they flow downward
4. Output the result at the bottom

You can find the example for this 2x2 matrix example here. The example accepts a `--speed` parameter so you can control how fast the visualization runs. Want to study each cycle carefully? Use `--speed 2000` for 2 seconds per cycle. Want a quick demo? Use `--speed 100`.

```rust
// https://github.com/Ayobami-00/tpu-matmul-simulator-rs/blob/main/examples/mat
use std::env;
use tpu_matmul_simulator_rs::{SystolicArray, systolic_matmul, naive_matmul};

fn main() {
    let args: Vec<String> = env::args().collect();
    let speed_ms = parse_speed(&args);
```

```
    // ... setup matrices A and B ...

    let array = SystolicArray::new(b.clone());
    let result = systolic_matmul(&array, &b, &a, speed_ms);

    // ... verify results ...
}
```

Running the Example

We've added a Makefile to make running examples easy:

```
# Run with default speed (500ms per cycle)
make run-2x2

# Run with custom speed (e.g., 200ms for faster, 2000ms for slower)
make run-2x2 SPEED=200

# Run without visualization (instant)
make run-2x2 SPEED=0

# Quick demo
make demo

# See all available commands
make help
```

You can also run directly with cargo:

```
cargo run --example matmul_2x2 -- --speed 300
```

What You'll See

When you run the example, you'll see the visualization update in real-time:

```
  Computing result row 1 of 2
```

```
+-------------------------------------------------------------------------------
|
|   SYSTOLIC ARRAY SIMULATOR                                  Cycle:  2 / 4
|
+===============================================================================
|
|                              PSUM IN (0 for top row)
|                                 [0.0]  [0.0]
|                                    |      |
|                                    v      v
|
|                             +---------+ +---------+
|                             | W:  5.0 | | W:  6.0 |
|                [  -  ] --> | A:  -   | | A:  1.0 | --> [  1.0]
|                             | P:  0.0 | | P:  6.0 |
|                             +---------+ +---------+
|                                    |           |
|                                    v           v
|                             +---------+ +---------+
|                             | W:  7.0 | | W:  8.0 |
|                [  -  ] --> | A:  2.0 | | A:  -   | --> [  -  ]
|                             | P: 19.0 | | P:  0.0 |
|                             +---------+ +---------+
|
|                                    |      |
|                                    v      v
|                                 [ 19.0]  [  0.0]
|                              PSUM OUT (result)
|
+===============================================================================
|   W = weight     A = activation     P = partial sum
|   Delay: 500ms/cycle
+-------------------------------------------------------------------------------
```

At cycle 2, you can see:

- Activation 1.0 has moved to PE(0,1), producing partial sum 6.0
- Activation 2.0 is at PE(1,0), and the partial sum 19.0 has accumulated (1×5 + 2×7 = 19)
- PSUM OUT shows 19.0 appearing at column 0 — that's C[0][0] !

The visualization clears and redraws each cycle, creating an animation effect. Watch the diagonal wavefront form as activations march right and partial sums accumulate downward.

At the end, the example compares the systolic array result against a naive CPU implementation:

```
Systolic array result:
  |   19.0   22.0 |
  |   43.0   50.0 |

Verification:
  ✓ Results match!
```

Comparing CPU vs Systolic

Let's step back and understand why systolic arrays matter. It's not about running faster on a simulator, it's about what happens in real hardware.

The CPU Approach

A traditional CPU computes matrix multiplication sequentially:

```
for i in 0..m {
    for j in 0..n {
        for k in 0..k {
            c[i][j] += a[i][k] * b[k][j];
        }
    }
}
```

For an N×N matrix multiplication:

- Operations: N^3 multiply-adds
- Memory accesses: Each operation needs to fetch operands from memory

- Parallelism: Limited by number of CPU cores (4–16 typically)

The Systolic Approach

A systolic array does the same math, but:

- Operations: Same N^3 multiply-adds total
- Memory accesses: Each weight loaded once, each activation flows through
- Parallelism: N^2 PEs working simultaneously

Note: The "CPU cycles" here assume one multiply–accumulate per cycle for conceptual comparison. Real CPUs use SIMD vector units and pipelining, but their overall throughput is still far below an N^2-PE systolic array and is ultimately bottlenecked by memory bandwidth.

The Numbers

Matrix Size	Total MACs	CPU (conceptual cycles)	Systolic Array	Spee
2×2	8	8 cycles	4 cycles (4 PEs)	2×
4×4	64	64 cycles	7 cycles (16 PEs)	9×
16×16	4,096	4,096 cycles	31 cycles (256 PEs)	13
256×256	16.7M	16.7M cycles	511 cycles (65K PEs)	32

Note: These CPU cycle counts are conceptual and assume one MAC per cycle; real CPUs use SIMD and caches, but are still bottlenecked by memory bandwidth, not compute units. The systolic array's key advantage remains: N^2 parallel MACs per cycle with explicit data reuse

The formula for systolic array cycles: `2N - 1` (time to fill the array + time to drain results).

This 2N−1 formula applies to a square N×N systolic array multiplying square matrices. The general latency for an M×N array processing a depth-K matmul is (M−1) + (N−1) + (K−1), which includes the wavefront fill, compute, and drain phases.

But Wait , Our Simulator Is Slower!

If you benchmark our Rust simulator, the “systolic” version is actually slower than naive matmul. Why?

Because we’re simulating parallelism, not achieving it.

Our simulator runs on a single CPU thread, stepping through each PE one at a time. The real benefit comes from hardware:

```
Real TPU:     65,536 PEs × 700 MHz = 92 trillion ops/sec
Our Simulator: 1 thread × sequential = just a simulation
```

What Real Hardware Gains

1. Massive parallelism — Thousands of PEs working simultaneously
2. Data reuse — Weights stay in place, activations stream through
3. Reduced memory bandwidth — The bottleneck in most AI workloads
4. Predictable timing — No cache misses, no branch mispredictions
5. Energy efficiency — Simple, repeated operations

The Memory Bandwidth Story

This is the real killer feature. For a 256×256 matmul:

CPU approach:

- Load A: 256×256 = 65K values
- Load B: 256×256 = 65K values
- For each of 16.7M operations, fetch from cache/memory
- Memory bandwidth is the bottleneck

Systolic approach:

- Load B once into the array: 65K values
- Stream A through: 65K values
- Each value used N times as it flows through
- N× reduction in memory traffic

This is why TPUs dominate inference workloads, they're designed around the memory hierarchy, not just raw compute.

The Takeaway

Our simulator proves the concept works. Real hardware proves it scales. The TPU's systolic array isn't magic , it's just matrix multiplication organized to maximize parallelism and minimize memory movement. Now you understand exactly how it works, one PE at a time.

What I Learned

Building this simulator taught me a lot. Here's what I learned:

1.Systolic Arrays Are Beautifully Simple: The core idea fits in one sentence: multiply-accumulate units arranged in a grid, with data flowing through in waves. No complex control logic, no caches, no branch predictors. Just data moving in lockstep.

2. The Wavefront Is Everything: I didn't truly understand systolic arrays until I watched the diagonal wavefront form in the visualization. Seeing activations march right while partial sums flow down , that's when it clicked. The staggered timing isn't a hack; it's the whole point. It's what creates the diagonal wavefront that lets an N×N array perform N^2 operations each cycle once the pipeline is full, and finish the entire matmul in roughly 2N cycles of latency.

3. Memory Bandwidth Is the Real Bottleneck: Before this project, I thought "more compute = faster." TPUs taught me that moving data costs more than computing on it.

The systolic array's genius is data reuse:

- Each weight is loaded from memory once per tile and reused across N MACs
- Each activation streams through, touching N PEs
- The result: on-chip, you get roughly an N× reduction in memory traffic compared to a naïve implementation.

4. Hardware Thinking Is Different: Software developers think in algorithms. Hardware developers think in data movement. When I first tried to implement matmul, I wrote it like software : loop over rows, loop over columns. It worked, but it missed the point. The systolic approach is about organizing computation so data flows naturally, minimizing movement and maximizing reuse.

Conclusion

We started with a question: *how does a TPU actually multiply matrices?*
Now we have an answer: not just in theory, but in working code. A 2D grid of processing elements, each doing one multiply-add per cycle, with data flowing through in synchronized waves. The systolic array isn't complicated. It's elegant. The same simple operation, repeated thousands of times in parallel, organized so data moves efficiently.

If you've followed along and run the examples, you now understand the computational heart of the hardware running most of the world's AI inference.

You can find the final full link to the codes used in this article on github. Clone it, run it, break it, extend it. You can also follow me on github and twitter. Looking forward to connecting and hearing how this article has helped you.

Till next time!

> *Thanks for reading. If you found this useful, check out the "Chewing on Chips" series that inspired it.*

Machine Learning | Computer Architecture | Rust Programming Language

Tensor Processing Unit | Deep Learning



Follow

Written by Owumi Festus

35 followers · 15 following

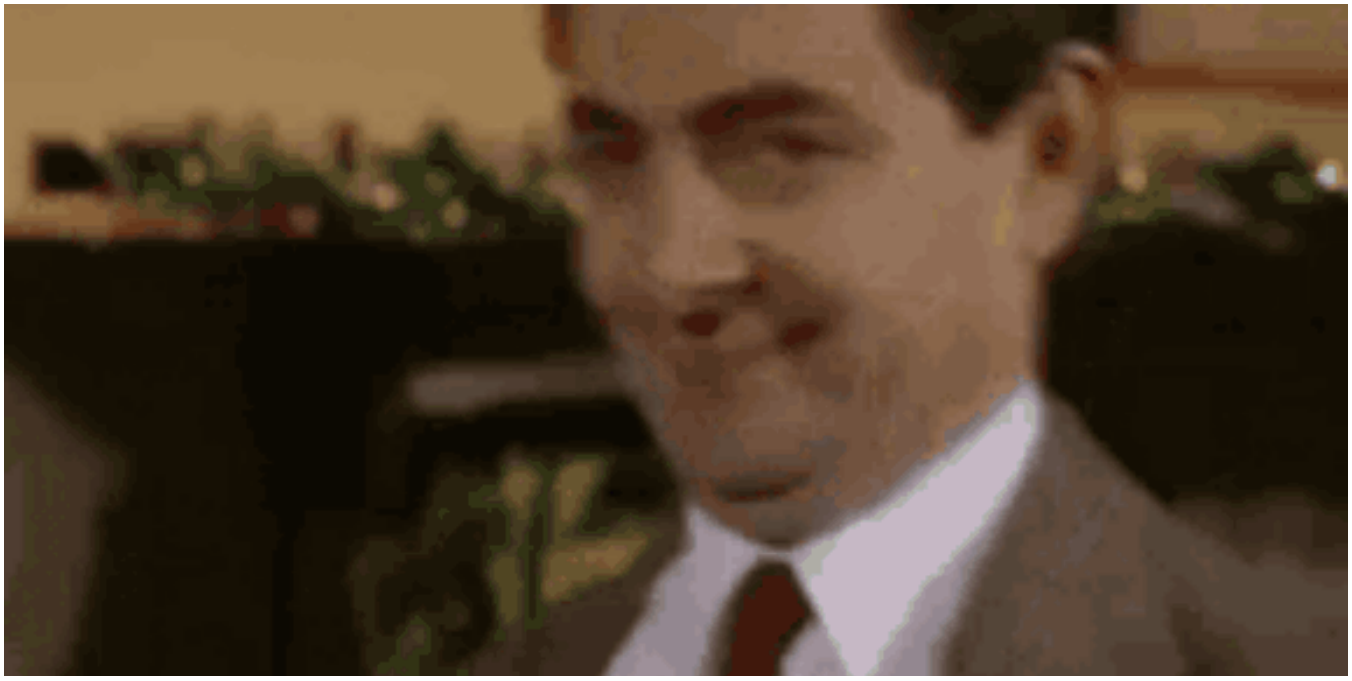
Software Engineer | Mobile | Backend | ML | Devops

No responses yet

Bala Vamsi Tatavarthy

What are your thoughts?

More from Owumi Festus

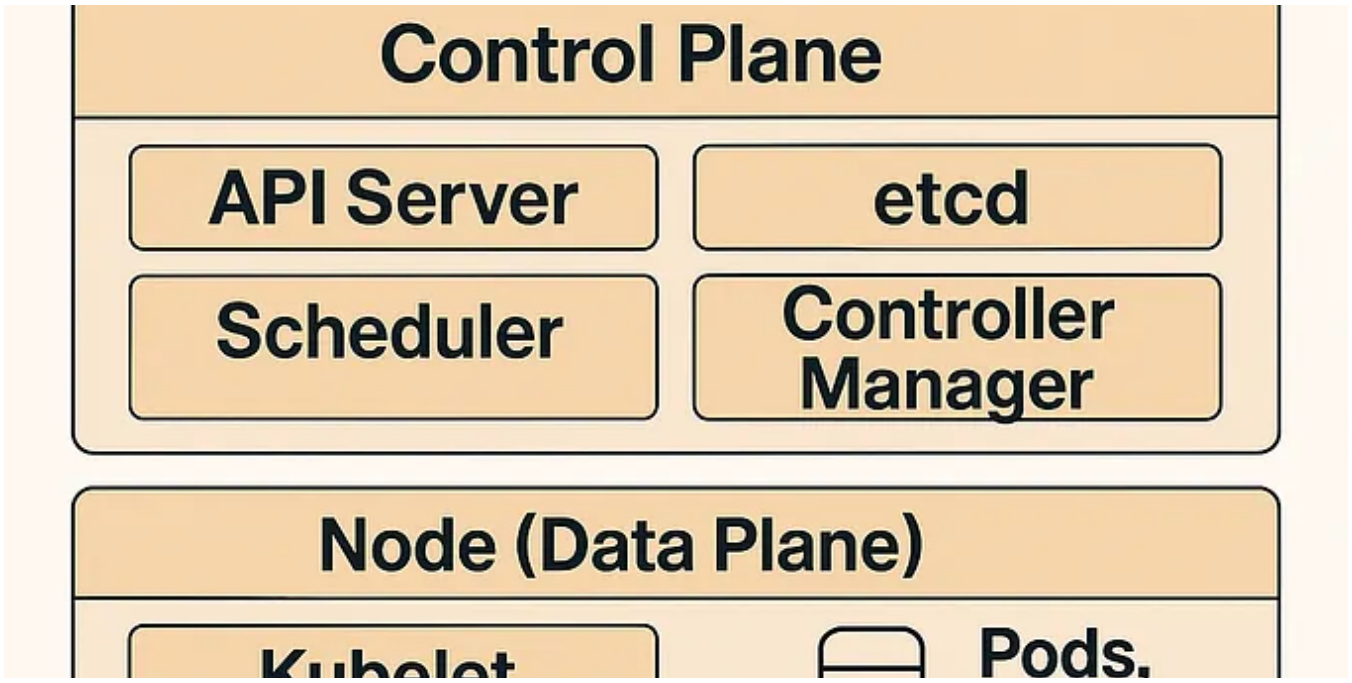


Owumi Festus

Automating Terraform Backend Setup: Bootstrapping S3 and DynamoDB State

Hello World!

Sep 16, 2024 5 1



Owumi Festus

Building Kubernetes (a lite version) from Scratch in Go

A hands-on journey to demystify Kubernetes by building a lite version of its control plane, scheduler and Kubelet in Go

May 10, 2025 130



Owumi Festus

Implementing a Multi Tenancy solution in a Kubernetes cluster

Hello World!

Mar 9, 2024 64



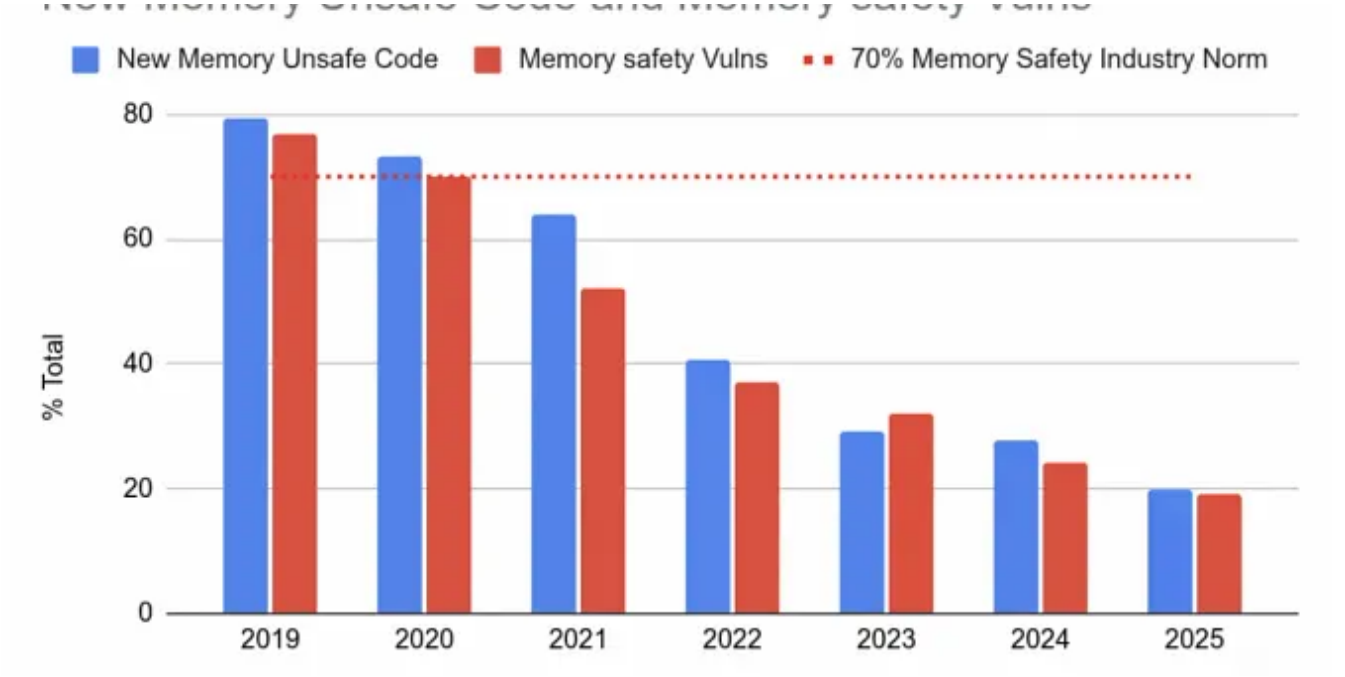
Owumi Festus

Automating Terraform Backend Setup: Bootstrapping Azure Storage

Feb 25, 2025

See all from Owumi Festus

Recommended from Medium



In CodeElevation by Dylan Cooper

Using Rust, Google's Real Test: Memory Vulnerability Rate on Android Is 1000× Lower Than C/C++!

In recent years, Rust has become a somewhat controversial language. On one hand, the U.S. government agencies publicly call for moving away...

5d ago 22 3



Joe Njenga

I Tried New Claude Code Ollama Workflow (It's Wild & Free)

Claude Code now works with Ollama, which takes the game to the next level for developers who want to work locally or need flexible model...

5d ago 899 15

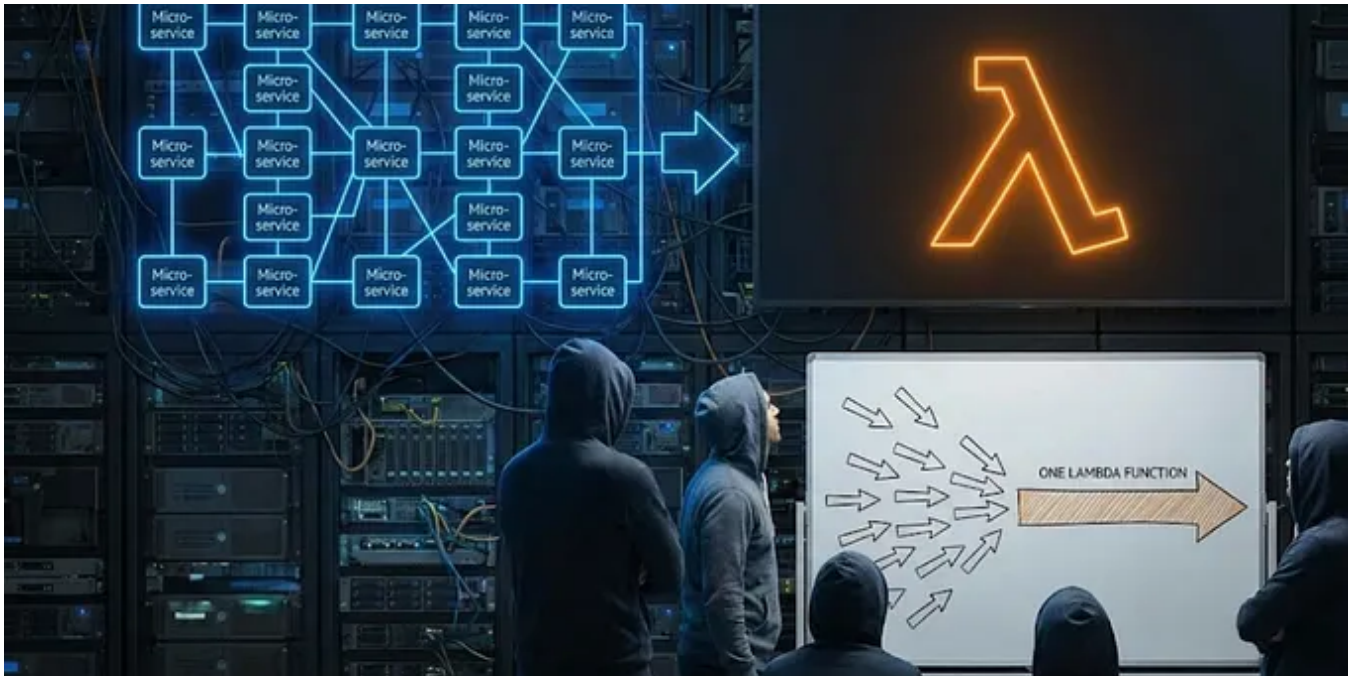


The Thread Whisperer

Go Vs Rust Vs Node: 1 Million Requests Later, The Fast One Lost

You shipped the fast stack because you were tired of explaining why p99 was someone else's problem.

3d ago 219 4



In AWS in Plain English by Sneha | DevOps | AWS | AI

We Replaced All Our Microservices With One Lambda Function.

Our “modern” microservices architecture was quietly destroying us.

5d ago 44 2

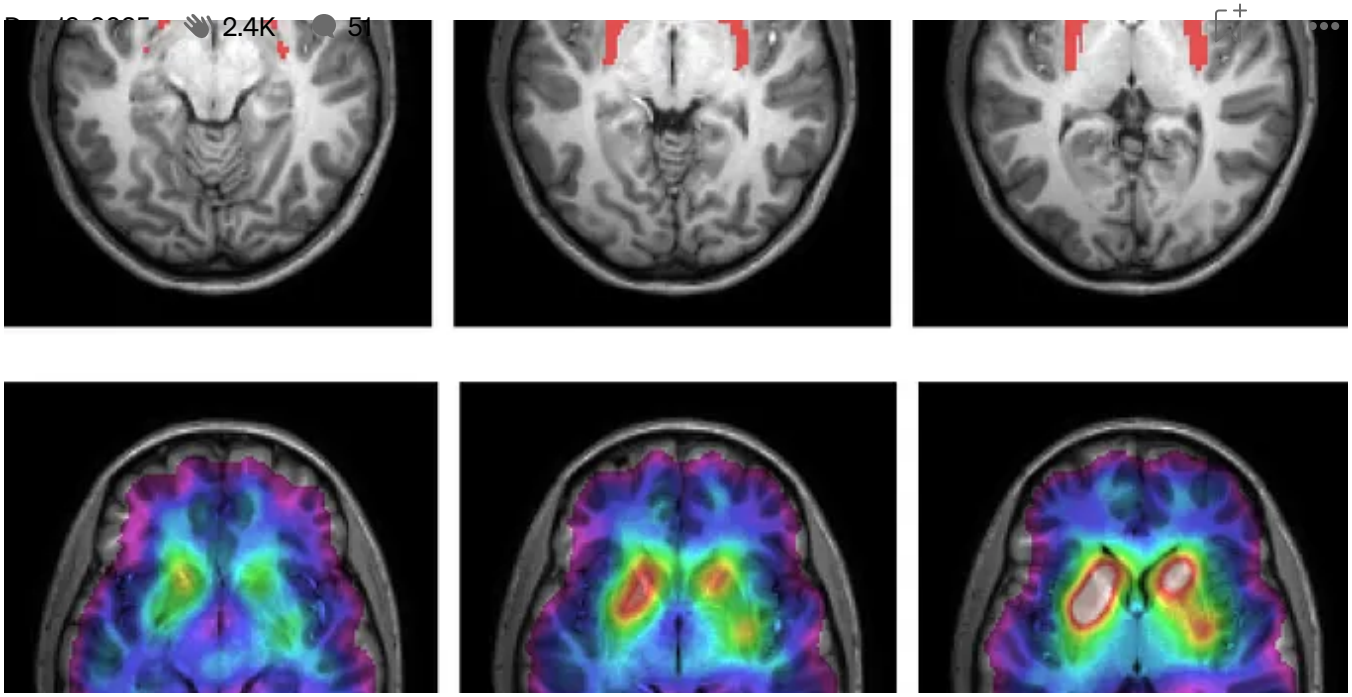


In Let's Code Future by The Unwritten Algorithm

Git Confused Me for Years Until I Found This Simple Guide

Most developers don't really understand Git — here's the simple truth.

2.4K 51



In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy Your Brain

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

Jan 15 15.5K 277

See more recommendations